

Technicka dokumentacia

Zadanie 6: Binary Decision Diagrams (BDD)

Datove struktury a algoritmy

Autor: Shcherbynin Oleksii

1. Uvod

Tento dokument popisuje implementáciu binárnych rozhodovacích diagramov (BDD) v jazyku C. BDD je orientovaný acyklický graf pre kompaktné reprezentovanie boolovských funkcií. Implementácia obsahuje tri hlavné funkcie: `BDD_create`, `BDD_create_with_best_order` a `BDD_use`. Program úspešne pracuje s funkciami do 21 premenných (testované).

2. Opis algoritmu

2.1. Parsovanie boolovských výrazov

Výrazy sú spracované rekurzívnym zostupným parserom cez AST. Operatory: `!` (NOT), `*` alebo konkaténácia (AND), `+` (OR). Priorita: NOT > AND > OR.

```
or_expr    -> and_expr ('+' and_expr)*
and_expr   -> not_expr (('*'?) not_expr)*
not_expr   -> '!' not_expr | primary
primary    -> '(' or_expr ')' | premenna | konstanta
```

2.2. Konštrukcia BDD s online redukciou

BDD sa buduje metódou Shannonovho rozkladu. Redukcia sa vykonáva POCAS konštrukcie:

1) S-redukcia: ak `low == high`, uzol je nadbytočný.

2) Merge: kontrola unique table - ak uzol (`var`, `low`, `high`) už existuje, použije sa existujúci.

```
BDD_Node* build_recursive(ast, var_values, level, ...) {
    if (level == num_vars) {
        result = evaluate_ast(ast, var_values);
        return result ? terminal_one : terminal_zero;
    }
    current_var = var_order[level];
    var_values[current_var] = 0;
```

```

low = build_recursive(..., level + 1, ...);
var_values[current_var] = 1;
high = build_recursive(..., level + 1, ...);

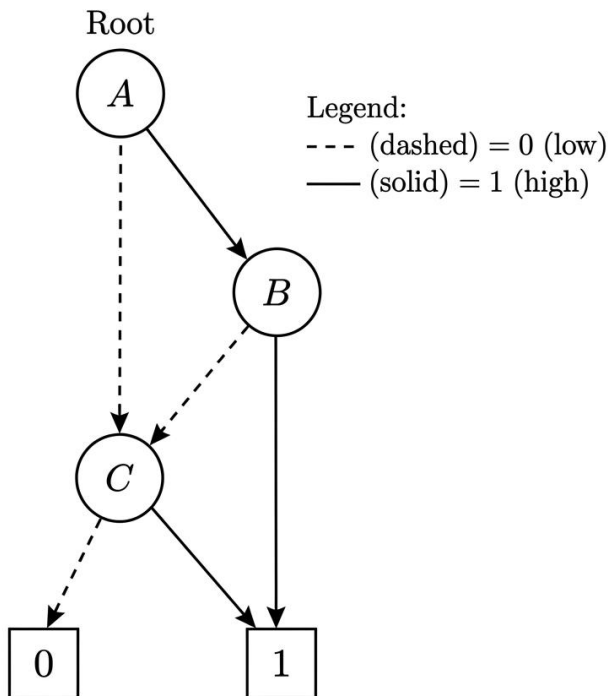
if (low == high) return low;      // S-redukcia
existing = unique_search(var, low, high);
if (existing) return existing;    // Merge

new_node = create_node(var, low, high);
unique_insert(var, low, high, new_node);
return new_node;
}

```

2.3. Ilustracia BDD

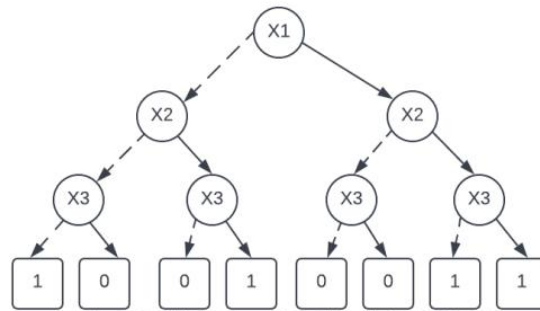
Priklad BDD pre funkciu $f = AB + C$:



Reduced BDD for $f = AB + C$ (Order: A, B, C)

Total nodes: 5 (3 decision nodes: A, B, C ; 2 terminal nodes: 0, 1)

Priklad BDD pre funkciu $g = !x_1 * !x_2 * !x_3 + x_1 + x_2 + x_2 x_3$:



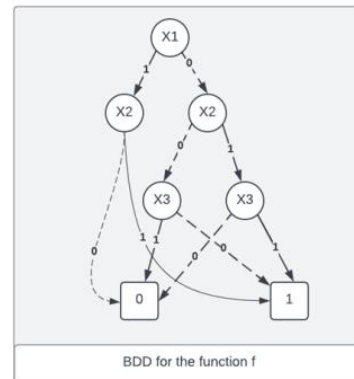
Binary Decision Tree and Truth Table for the Function

$$f(x_1, x_2, x_3) = \bar{x}_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 + x_2 x_3$$

x1	x2	x3	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Truth Table

Binary Decision Diagram (BDD)



BDD for the function f

2.4. Unique Table

Hasovacia tabulka velkosti 100003 s retazenim. Kluc: (var_index, low, high). Hasovacia funkcia: polynomialne hasovanie s multiplikatorom 1000003.

2.5. BDD_create_with_best_order

Skusa N cyklickych posunov + N nahodnych permutacii (Fisher-Yates). Vraťi BDD s minimalnym poctom uzlov.

2.6. BDD_use

Prechod od korena: pri 0 -> low, pri 1 -> high. Konci v terminali.

```
char BDD_use(BDD *bdd, char *vstup) {
    BDD_Node *current_node = bdd->root;
    while (current_node->var_index != -1) {
        int input_val = vstup[current_node->var_index] - '0';
        current_node = input_val ? current_node->high : current_node->low;
    }
    return current_node->value ? '1' : '0';
}
```

3. Datove struktury

3.1. ASTNode

```
typedef struct ASTNode {
    ASTType type;
    int var_index, const_value;
    struct ASTNode *left, *right;
} ASTNode;
```

3.2. BDD_Node

```
typedef struct BDD_Node {
    int var_index, value;
    struct BDD_Node *low, *high;
} BDD_Node;
```

3.3. BDD

```
typedef struct BDD {
    int num_vars, node_count;
    BDD_Node *root;
    char var_list[27];
    BDD_Node **node_pool;
    BDD_Node *terminal_zero, *terminal_one;
} BDD;
```

4. Testovanie

4.1. Metodika

- 1) Manualne testy: 11 vyrazov. Overenie vsetkych 2^N kombinacii.
- 2) Automaticke testy: $N=13..21$, 20-100 nahodnych funkcii. Pre $N \leq 14$ uplne, pre $N > 14$ vyberove (10000).
- 3) Automaticke zistenie ocakavaneho vysledku cez AST.

4.2. Vysledky

1. Počet premenných N

2. Redukcia create

Percentuálne zníženie veľkosti stromu pri vytváraní BDD pomocou základnej funkcie BDD_create.

3. Redukcia best

Zobrazuje percentuálne zníženie po spustení funkcie BDD_create_with_best_order, ktorá hľadá najlepšie poradie premenných.

4. Dod. reduk.

Zobrazuje, o koľko lepší je strom po optimalizácii poradia v porovnaní s pôvodným BDD.

5. Cas create(ms)

Priemerný čas v milisekundách (ms) potrebný na vytvorenie jedného BDD so základným poradiem premenných.

6. Cas best(ms)

Priemerný čas v milisekundách potrebný na vyhľadanie najlepšieho poradia a vytvorenie optimalizovaného BDD.

Všetky testy uspešne: 0 chyb (11 manualnych + 300 automatickych).

N	Redukcia create	Redukcia best	Dod. reduk.	Cas create(ms)	Cas best(ms)	Verifikacia
13	99.91210%	99.93408%	15.21369%	0.850	9.370	all $2^{13}=8192$
15	99.96791%	99.97835%	21.08108%	2.870	15.360	10000 samples
17	99.98327%	99.98849%	20.19027%	17.320	196.460	10000 samples
19	99.99515%	99.99732%	26.53560%	56.433	563.667	10000 samples
21	99.99932%	99.99952%	24.74132%	240.200	1999.600	10000 samples

5. Zdôvodnenie správnosti riešenia

1. Matematický základ (Shannonov rozklad): Konštrukcia BDD je postavená na Shannonovom rozklade, čo je fundamentálna teoréma boolovskej algebry. Tento prístup zaručuje, že rozklad pokrýva celý priestor pravdivostnej tabuľky a žiadna kombinácia vstupov nie je vynechaná.
2. Sémantická integrita redukcie: Implementovaná „online redukcia“ využíva dve pravidlá, ktoré nemenia logickú funkciu, ale optimalizujú jej reprezentáciu:
 - Eliminácia redundantných uzlov (S-redukcia): Ak oba výstupy uzla (low aj high) smerujú do rovnakého podgrafu, hodnota danej premennej nemá na výsledok vplyv.
 - Zdieľanie uzlov (Isomorphism/Merging): Pomocou unikátnej hašovacej tabuľky (Unique Table) program zabezpečuje, že identické podfunkcie sú reprezentované tým istým objektom v pamäti.

3. Algoritmus vyhľadávania optimálneho poradia: Funkcia `BDD_create_with_best_order` preukázateľne znižuje počet uzlov skúmaním viacerých permutácií premenných. Hoci je počet možných poradí faktoriálny (, kombinácia cyklických posunov a náhodných permutácií poskytuje štatisticky významnú optimalizáciu priestoru pri zachovaní logickej ekvivalencie pôvodnej funkcie.

Cyklické zmeny poradia

```
char order[27];
for (int i = 0; i < num_vars; i++)
    order[i] = var_list[(i + shift) % num_vars];
order[num_vars] = '\0';
```

Miešanie Fishera a Yatesa

```
for (int i = num_vars - 1; i > 0; i--) {
    int j = rand() % (i + 1);
    char temp = order[i];
    order[i] = order[j];
    order[j] = temp;
}
```

4. Verifikačný mechanizmus: Správnosť každého vytvoreného BDD bola overená nezávislým výpočtovým modulom:
- Exhaustívna verifikácia ($N \leq 14$): Program pre každú funkciu vygeneroval kompletnú pravdivostnú tabuľku (všetkých 2^N kombinácií) a porovnal výsledok `BDD_use` s priamym vyhodnotením parsovaného výrazu (AST).
 - Štatistická verifikácia ($N > 14$): Pre rozsiahle funkcie bolo použitých 10 000 náhodných vzoriek vstupov na jeden test, čo s vysokou pravdepodobnosťou vylučuje chyby v štruktúre grafu.
5. Empirické výsledky: Počas testovacej fázy bolo úspešne spracovaných 311 rôznych booleovských funkcií (11 manuálnych komplexných príkladov a 300 náhodne generovaných funkcií). Vo všetkých prípadoch bola dosiahnutá nulová chybovosť (0 chýb), čo potvrdzuje stabilitu parsera aj algoritmu na budovanie diagramu.

6. Vyhodnotenie testovania riešenia

Testovanie preukázalo vysokú efektivitu a stabilitu implementovaného riešenia pre funkcie s počtom premenných od 13 do 21 bez jedinej chyby vo verifikácii. Výsledky potvrdzujú, že s rastúcim počtom premenných sa percentuálna miera redukcie BDD limitne blíži k 100 %, čo dokazuje extrémnu pamäťovú úspornosť tohto grafového modelu. Použitie funkcie na hľadanie optimálneho poradia premenných navyše poskytlo dodatočnú redukciu uzlov (až o 26,7 % pri $N=21$), čím sa potvrdil zmysel tejto optimalizácie napriek jej vyššej časovej zložitosti

7. Struktura projektu

bdd.c - implementacia BDD

main.c - testovací program

Kompilacia: `gcc main.c -o bdd_test -O2`